9
Graph Traversal

Graphs are one of the unifying themes of computer science – an abstract representation which describes the organization of transportation systems, electrical circuits, human interactions, and telecommunication networks. That so many different structures can be modeled using a single formalism is a source of great power to the educated programmer.

In this chapter, we focus on problems which require only an elementary knowledge of graph algorithms, specifically the appropriate use of graph data structures and traversal algorithms. In Chapter 10, we will present problems relying on more advanced graph algorithms that find minimum spanning trees, shortest paths, and network flows.

9.1 Flavors of Graphs

A graph $G = (V, E)$ is defined by a set of *vertices* V, and a set of *edges* E consisting of ordered or unordered pairs of vertices from V. In modeling a road network, the vertices may represent the cities or junctions, certain pairs of which are connected by roads/edges. In analyzing the source code of a computer program, the vertices may represent lines of code, with an edge connecting lines x and y if y can be the next statement executed after x. In analyzing human interactions, the vertices typically represent people, with edges connecting pairs of related souls.

There are several fundamental properties of graphs which impact the choice of data structures used to represent them and algorithms available to analyze them. The first step in any graph problem is determining which flavor of graph you are dealing with:

- *Undirected vs. Directed* — A graph $G = (V, E)$ is *undirected* if edge $(x, y) \in E$ implies that (y, x) is also in E. If not, we say that the graph is *directed*. Road

networks *between* cities are typically undirected, since any large road has lanes going in both directions. Street networks *within* cities are almost always directed, because there are typically at least a few one-way streets lurking about. Program-flow graphs are typically directed, because the execution flows from one line into the next and changes direction only at branches. Most graphs of graph-theoretic interest are undirected.

- *Weighted vs. Unweighted* — In *weighted* graphs, each edge (or vertex) of G is assigned a numerical value, or weight. Typical application-specific edge weights for road networks might be the distance, travel time, or maximum capacity between x and y. In *unweighted* graphs, there is no cost distinction between various edges and vertices.

  The difference between weighted and unweighted graphs becomes particularly apparent in finding the shortest path between two vertices. For unweighted graphs, the shortest path must have the fewest number of edges, and can be found using the breadth-first search algorithm discussed in this chapter. Shortest paths in weighted graphs requires more sophisticated algorithms, discussed in Chapter 10.

- *Cyclic vs. Acyclic* — An *acyclic* graph does not contain any cycles. *Trees* are connected acyclic *undirected* graphs. Trees are the simplest interesting graphs, and inherently recursive structures since cutting any edge leaves two smaller trees.

  Directed acyclic graphs are called *DAGs*. They arise naturally in scheduling problems, where a directed edge (x, y) indicates that x must occur before y. An operation called *topological sorting* orders the vertices of a DAG so as to respect these precedence constraints. Topological sorting is typically the first step of any algorithm on a DAG, and will be discussed in Section 9.5.

- *Simple vs. Non-simple* — Certain types of edges complicate the task of working with graphs. A *self-loop* is an edge (x, x) involving only one vertex. An edge (x, y) is a *multi-edge* if it occurs more than once in the graph.

  Both of these structures require special care in implementing graph algorithms. Hence any graph which avoids them is called *simple*.

- *Embedded vs. Topological* — A graph is *embedded* if the vertices and edges have been assigned geometric positions. Thus any drawing of a graph is an embedding, which may or may not have algorithmic significance.

  Occasionally, the structure of a graph is completely defined by the geometry of its embedding. For example, if we are given a collection of points in the plane, and seek the minimum cost tour visiting all of them (i.e., the traveling salesman problem), the underlying topology is the *complete graph* connecting each pair of vertices. The weights are typically defined by the Euclidean distance between each pair of points.

  Another example of topology from geometry arises in grids of points. Many problems on an $n \times m$ grid involve walking between neighboring points, so the edges are implicitly defined from the geometry.

- *Implicit vs. Explicit* — Many graphs are not explicitly constructed and then traversed, but built as we use them. A good example is in backtrack search. The vertices of this implicit search graph are the states of the search vector, while edges link pairs of states which can be directly generated from each other. It is often easier to work with an implicit graph than explicitly constructing it before analysis.

- *Labeled vs. Unlabeled* — In *labeled* graphs, each vertex is assigned a unique name or identifier to distinguish it from all other vertices. In *unlabeled* graphs, no such distinctions have been made.

  Most graphs arising in applications are naturally and meaningfully labeled, such as city names in a transportation network. A common problem arising on graphs is that of *isomorphism testing*, determining whether the topological structure of two graphs are in fact identical if we ignore any labels. Such problems are typically solved using backtracking, by trying to assign each vertex in each graph a label such that the structures are identical.

9.2 Data Structures for Graphs

There are several possible ways to represent graphs. We discuss four useful representations below. We assume the graph $G = (V, E)$ contains n vertices and m edges.

- *Adjacency Matrix* — We can represent G using an $n \times n$ matrix M, where element $M[i, j]$ is, say, 1, if (i, j) is an edge of G, and 0 if it isn't. This allows fast answers to the question "is (i, j) in G?", and rapid updates for edge insertion and deletion. It may use excessive space for graphs with many vertices and relatively few edges, however.

  Consider a graph which represents the street map of Manhattan in New York City. Every junction of two streets will be a vertex of the graph, with neighboring junctions connected by edges. How big is this graph? Manhattan is basically a grid of 15 avenues, each crossing roughly 200 streets. This gives us about 3,000 vertices and 6,000 edges, since each vertex neighbors four other vertices and each edge is shared between two vertices. Such a small amount of data should easily and efficiently stored, but the adjacency matrix will have $3{,}000 \times 3{,}000 = 9{,}000{,}000$ cells, almost all of them empty!

- *Adjacency Lists in Lists* — We can more efficiently represent sparse graphs by using linked lists to store the neighbors adjacent to each vertex. Adjacency lists require pointers but are not frightening once you have experience with linked structures.

  Adjacency lists make it harder to ask whether a given edge (i, j) is in G, since we have to search through the appropriate list to find the edge. However, it is often surprisingly easy to design graph algorithms which avoid any need for such queries. Typically, we sweep through all the edges of the graph in one pass via a

breadth-first or depths-first traversal, and update the implications of the current edge as we visit it.

- *Adjacency Lists in Matrices* — Adjacency lists can also embedded in matrices, thus eliminating the need for pointers. We can represent a list in an array (or equivalently, a row of a matrix) by keeping a count k of how many elements there are, and packing them into the first k elements of the array. Now we can visit successive the elements from the first to last just like a list, but by incrementing an index in a loop instead of cruising through pointers.

  This data structure looks like it combines the worst properties of adjacency matrices (large space) with the worst properties of adjacency lists (the need to search for edges). However, there is a method to its madness. First, it is the simplest data structure to program, particularly for static graphs which do not change after they are built. Second, the space problem can in principle be eliminated by allocating the rows for each vertex dynamically, and making them exactly the right size.

  To prove our point, we will use this representation in all our examples below.

- *Table of Edges* — An even simpler data structure is just to maintain an array or linked list of the edges. This is not as flexible as the other data structures at answering "who is adjacent to vertex x?" but it works just fine for certain simple procedures like Kruskal's minimum spanning tree algorithm.

As stated above, we will use adjacency lists in matrices as our basic data structure to represent graphs. It is not complicated to convert these routines to honest pointer-based adjacency lists. Sample code for adjacency lists and matrices can be found in many books, including [Sed01].

We represent a graph using the following data type. For each graph, we keep count of the number of vertices, and assign each vertex a unique number from 1 to `nvertices`. We represent the edges in an `MAXV` × `MAXDEGREE` array, so each vertex can be adjacent to `MAXDEGREE` others. By defining `MAXDEGREE` to be `MAXV`, we can represent any simple graph, but this is wasteful of space for low-degree graphs:

```
#define MAXV            100             /* maximum number of vertices */
#define MAXDEGREE       50              /* maximum vertex outdegree */

typedef struct {
        int edges[MAXV+1][MAXDEGREE];   /* adjacency info */
        int degree[MAXV+1];             /* outdegree of each vertex */
        int nvertices;                  /* number of vertices in graph */
        int nedges;                     /* number of edges in graph */
} graph;
```

We represent a directed edge (x, y) by the integer y in x's adjacency list, which is located in the subarray `graph->edges[x]`. The degree field counts the number of meaningful entries for the given vertex. An undirected edge (x, y) appears twice in any adjacency-based graph structure, once as y in x's list, and once as x in y's list.

To demonstrate the use of this data structure, we show how to read in a graph from a file. A typical graph format consists of an initial line featuring the number of vertices and edges in the graph, followed by a listing of the edges at one vertex pair per line.

```
read_graph(graph *g, bool directed)
{
        int i;                          /* counter */
        int m;                          /* number of edges */
        int x, y;                       /* vertices in edge (x,y) */

        initialize_graph(g);

        scanf("%d %d",&(g->nvertices),&m);

        for (i=1; i<=m; i++) {
                scanf("%d %d",&x,&y);
                insert_edge(g,x,y,directed);
        }
}

initialize_graph(graph *g)
{
        int i;                          /* counter */

        g -> nvertices = 0;
        g -> nedges = 0;

        for (i=1; i<=MAXV; i++) g->degree[i] = 0;
}
```

The critical routine is `insert_edge`. We parameterize it with a Boolean flag `directed` to identify whether we need to insert two copies of each edge or only one. Note the use of recursion to solve the problem:

```
insert_edge(graph *g, int x, int y, bool directed)
{
        if (g->degree[x] > MAXDEGREE)
            printf("Warning: insertion(%d,%d) exceeds max degree\n",x,y);

        g->edges[x][g->degree[x]] = y;
        g->degree[x] ++;

        if (directed == FALSE)
                insert_edge(g,y,x,TRUE);
        else
                g->nedges ++;
```